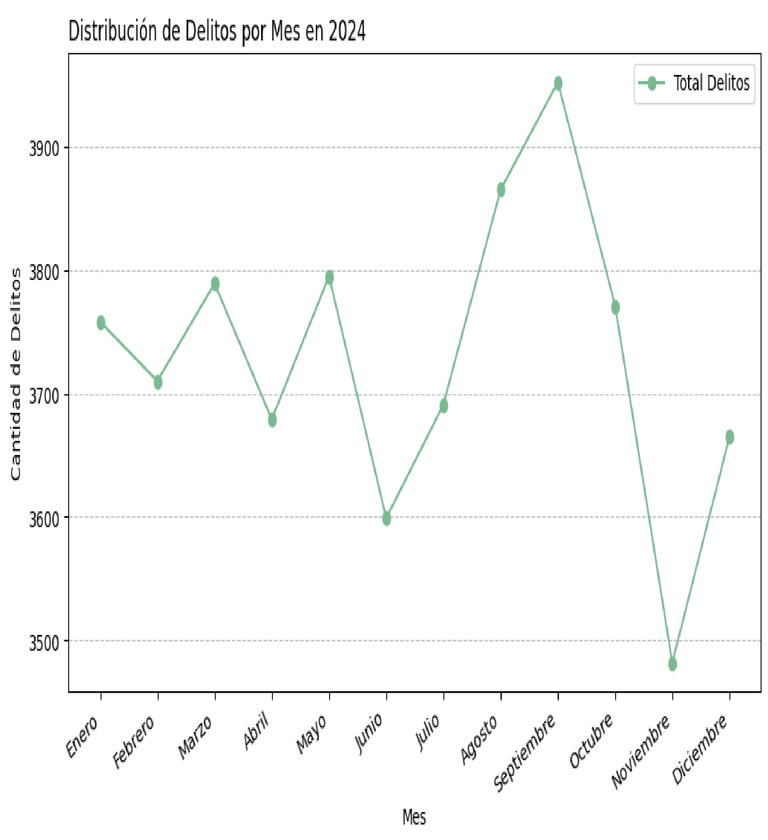
Gráficos Estadísticos: Lineal y Araña

Sebastián Calvo Solano, Isaac Rodríguez
Colegio Universitario de Cartago

Gráfico Lineal

El gráfico lineal permite mostrar tendencias o evolución en el tiempo, estas están conectadas por líneas, muestra tendencia o patrones y permite comparar múltiples series simultáneamente.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico



Código

Código utilizado realizar el gráfico en Python:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

datos["Fecha"] = pd.to_datetime(datos["Fecha"], errors='coerce') #Filtra los datos por fecha
datos['Nombre_Mes'] = datos["Fecha"].dt.month_name(locale='es_ES') #Asigna un nombre al mes (solo están las fechas)
datos['Numero_Mes'] = datos["Fecha"].dt.month #Asigna el número a el mes
datos_limpios = datos.dropna(subset=['Nombre_Mes'])

conteo_mensual = datos_limpios.groupby(['Numero_Mes', 'Nombre_Mes']).size().reset_index(name='Total_Delitos') #Asigna los casos que hay a los meses
conteo_mensual = conteo_mensual.sort_values(by='Numero_Mes') #cuentan el total

fig, ax = plt.subplots(figsize=(10, 6)) #definición del gráfico

plt.plot('Nombre_Mes', 'Total_Delitos',
         data=conteo_mensual, #de donde obtienen la información
         linestyle='-', #la líena que van a utilizar
         marker='o', #para marcar cual mes es
         color='#7bba91', #color
         label='Total Delitos') #la división que se hace con el eje y


plt.title("Distribución de Delitos por Mes en 2024", loc='left') #Título
plt.xlabel("Mes") #leyenda del eje x
plt.ylabel("Cantidad de Delitos") #leyenda eje y

fig.autofmt_xdate(rotation=35)

plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend()
plt.show()
```

Gráfico de Araña o Radar

El gráfico de araña o radar permite comparar múltiples en un formato radial todas las variablies surgen de un punto central y permite mostrar fortalezas y debilidades de forma visual.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico y Código




```python
import pandas as pd
import matplotlib.pyplot as plt
from math import pi

link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

provincias_radar = ["SAN JOSE", "HEREDIA", "CARTAGO"]
datos_radar = datos[datos["Provincia"].isin(provincias_radar)].copy()

# Crear tabla cruzada para contar delitos por provincia
df_radar = pd.crosstab(datos_radar['Provincia'], datos_radar['Delito'])
df_radar = df_radar.T # Transponer para que los delitos sean las filas (ejes del radar)

# 1. Normalizar los datos: Escalar los valores entre 0 y 1 para que el gráfico sea comparable
df_normalized = (df_radar - df_radar.min()) / (df_radar.max() - df_radar.min())
categories = df_normalized.index.tolist()
N = len(categories)

# 2. Preparar el ángulo para cada eje
angles = [n / float(N) * 2 * pi for n in range(N)]
angles += angles[:1] # Cierra el círculo

fig, ax = plt.subplots(figsize=(10, 10), subplot_kw=dict(polar=True))

# 3. Dibujar cada provincia
for i, provincia in enumerate(provincias_radar):
    values = df_normalized[provincia].values.flatten().tolist()
    values += values[:1] # Cierra la línea

    # Dibujar la línea y el área
    color = ["#006a5e", "#559f80", "#a2d2a4"][i]
    ax.plot(angles, values, linewidth=2, linestyle='solid', label=provincia, color=color)
    ax.fill(angles, values, color=color, alpha=0.2)

# 4. Configuración del gráfico
ax.set_xticks(angles[:-1])
ax.set_xticklabels(categories, size=8) # Etiquetas de los delitos
ax.set_yticks([0.25, 0.50, 0.75, 1.00]) # Puntos de la escala
ax.set_yticklabels(["25%", "50%", "75%", "100%"], color='grey', size=7)
ax.set_ylim(0, 1)

plt.title("Gráfico de Araña: Comparativa de Delitos Normalizados (San José, Heredia, Cartago)", size=12, y=1.1, loc='left')
plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
plt.show()
```

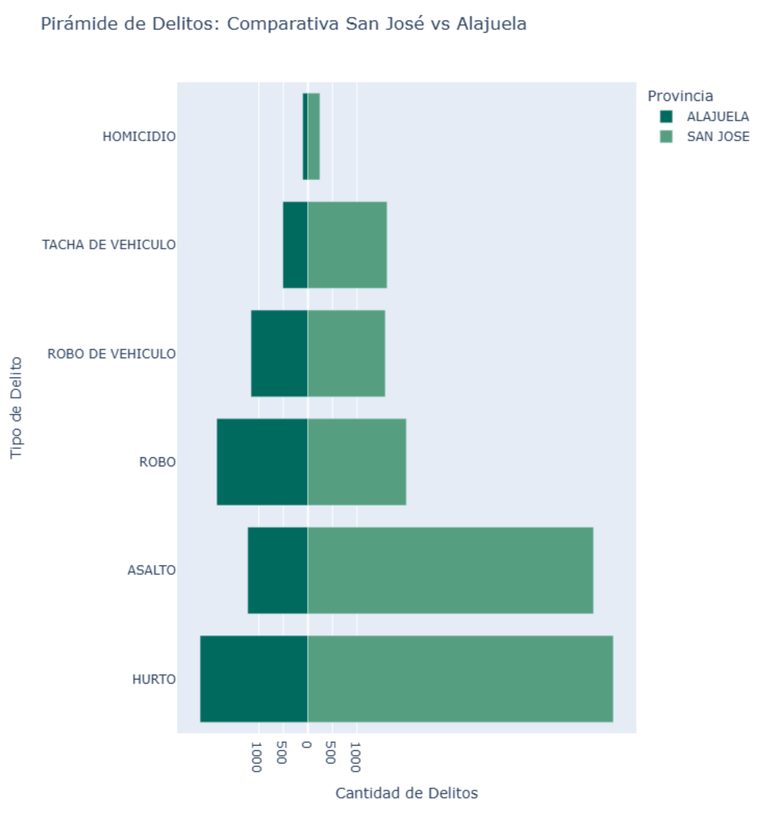
Gráficos Estadísticos: Pirámide y Pastel

Sebastián Calvo Solano, Isaac Rodríguez
Colegio Universitario de Cartago

Gráfico de Pirámide

El gráfico de pirámide permite comparar dos grupos que se oponen o complementan, estas barras parten de un eje central, permite ver la simetría o asimetría y cada eje es una variable.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico



Código

Código utilizado realizar el gráfico en Python:

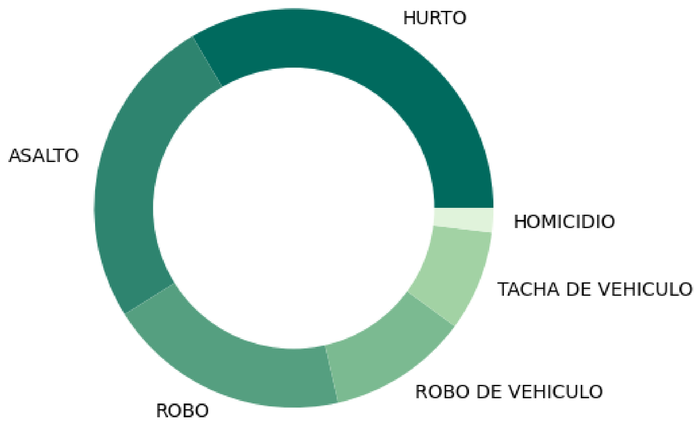
```python
import pandas as pd
import plotly.express as px

link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)
provincias_a_comparar = ["SAN JOSE", "ALAJUELA"] #Filtro para solo inlcuir a Alajuela y San José
datos_comparativa = datos[datos["Provincia"].isin(provincias_a_comparar)].copy() #Introducir el filtro
# Contar la ocurrencia de cada Delito en cada Provincia
delitos_conteo = datos_comparativa.groupby(['Delito', 'Provincia']).size().reset_index(name='Total_Delitos')
# Obtener el orden de los delitos por su frecuencia total (de menor a mayor)
orden_frecuencia = delitos_conteo.groupby('Delito')['Total_Delitos'].sum().sort_values(ascending=True).index.tolist()
# Convertir el conteo de Alajuela a negativo para la visualización de pirámide
delitos_conteo['Total_Delitos_Piramide'] = delitos_conteo.apply(
    lambda row: row['Total_Delitos'] * -1 if row['Provincia'] == 'ALAJUELA' else row['Total_Delitos'], axis=1
)
# Usar Plotly para crear la pirámide
fig = px.bar(
    delitos_conteo,
    x='Total_Delitos_Piramide',
    y='Delito',
    color='Provincia',
    orientation='h',
    height=800,
    title="Pirámide de Delitos: Comparativa San José vs Alajuela",
    color_discrete_sequence=['#006a5e', '#559f80'],
    # Aplicar el orden de frecuencia
    category_orders={"Delito": orden_frecuencia}
)
fig.update_layout(
    barmode='relative',
    xaxis=dict(
        # Ajustar los ticks para un rango simétrico y centrado en cero
        tickvals=[-1000, -500, 0, 500, 1000],
        ticktext=['1000', '500', '0', '500', '1000'],
        title='Cantidad de Delitos'
    ),
    yaxis=dict(title='Tipo de Delito')
)
fig.update_traces(
    hovertemplate="<b>Provincia:</b> %{customdata[0]}<br>" +
                  "<b>Delitos:</b> %{x|:,.0f}<br>" +
                  "<extra></extra>",
    customdata=delitos_conteo[['Provincia']]
)
fig.show()
```

Gráfico de Pastel

El gráfico de pastel (o en este caso dona) se utiliza para mostrar proporciones de un total, esto muestra las partes de un todo, idealmente utilizar entre 5-7 categorías máximo y cada segmentos es proporcional a su porcentaje, su órden es de mayor a menor en sentido del reloj.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico y Código




```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

#link de datos
link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

filtro_incluir = (datos["Provincia"] == "SAN JOSE") #Filtro para solo inlcuir a San José
filtro = datos["Provincia"] != "DESCONOCIDO" #los valores se transforman en booleanos y excluye DESCONOCIDO
datos_filtrados = datos[filtro] #los valores se convierten en un data frame
delitos = datos_filtrados['Delito'].value_counts()
orden_delitos = delitos.index.tolist()

my_circle = plt.Circle( (0,0), 0.7, color='white') #creación del círculo blanco del medio


plt.pie(delitos, labels=orden_delitos, colors= ["#006a5e", "#2e846f", "#559f80", "#7bba91", "#a2d2a4", "#e0f3db"])
#división de colores
p = plt.gcf()
p.gca().add_artist(my_circle)
```

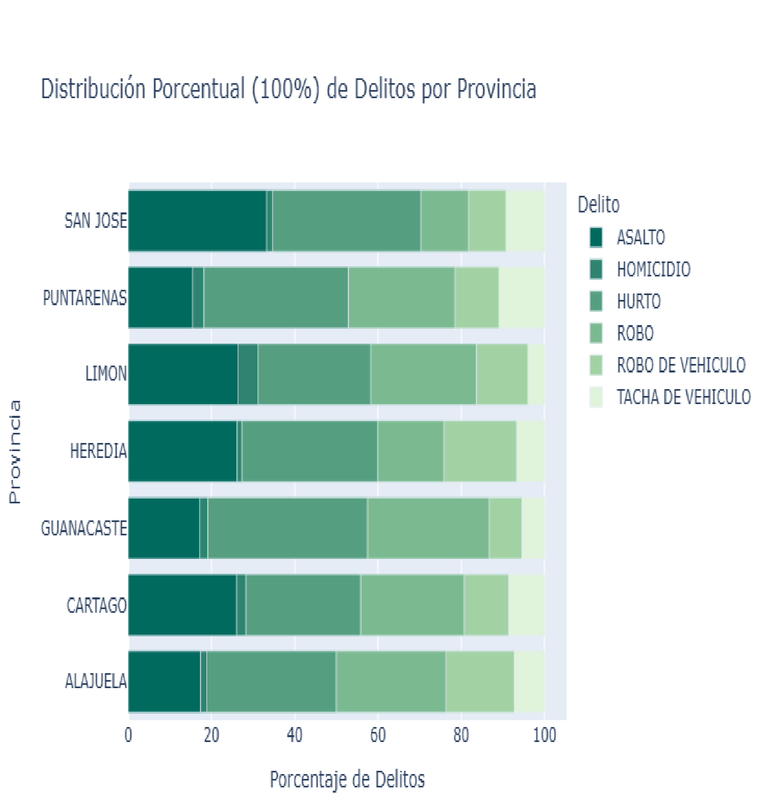
Gráficos Estadísticos: Barras 100 y Comparativas

Sebastián Calvo Solano, Isaac Rodríguez
Colegio Universitario de Cartago

Barra 100

El gráfico de barras 100 se utiliza para comparar la composición porcentual de un objeto, todas las barras poseen la misma altura facilita comparar proporciones entre grupos, este es similiar a las barras compuestas pero estandarizado.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico



Código

Código utilizado realizar el gráfico en Python:

```python
import pandas as pd
import plotly.express as px

link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

filtro = datos["Provincia"] != "DESCONOCIDO"
datos_filtrados = datos[filtro]

# Crea una tabla de frecuencias de Delito por Provincia
df_crosstab = pd.crosstab(datos_filtrados['Provincia'], datos_filtrados['Delito'])

# Calcula el porcentaje: divide cada valor entre la suma total de su fila (axis=1)
df_percent = df_crosstab.apply(lambda x: x / x.sum(), axis=1)
df_percent = df_percent.stack().reset_index(name='Porcentaje')
df_percent['Porcentaje'] = df_percent['Porcentaje'] * 100 # Convertir a %

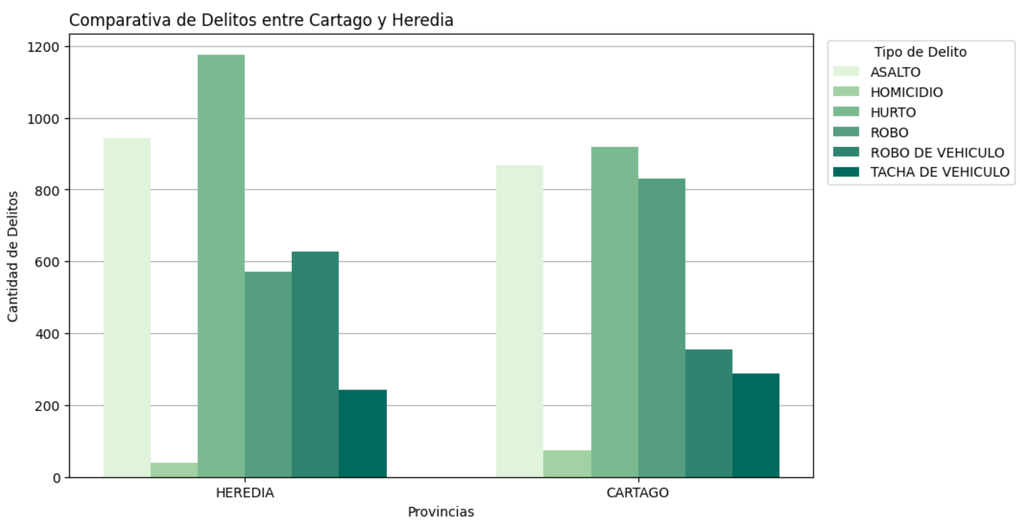
# Usa Plotly para un gráfico de barras 100% (con mode='percent')
fig = px.bar(
    df_percent,
    x='Porcentaje', #eje x
    y='Provincia', #eje y
    color='Delito', #el color se va a mostrar en relación a los delitos
    orientation='h', #orientación "h" para decir que es horizontal
    height=500, #tamaño de las barras
    title="Distribución Porcentual (100%) de Delitos por Provincia", #título del gráfico
    color_discrete_sequence=["#006a5e", "#2e846f", "#559f80", "#7bba91", "#a2d2a4", "#e0f3db"], #colores que se van a mostrar
    category_orders={"y": df_percent['Provincia'].unique().tolist()} #ordenar las barras por provincia (que está acomodado por delitos)
)

#contenido del hover
fig.update_layout(xaxis_title='Porcentaje de Delitos', yaxis_title='Provincia')
fig.update_traces(
    hovertemplate="<b>Provincia:</b> %{y}<br>" +
                  "<b>Delito:</b> %{customdata[0]}<br>" +
                  "<b>Porcentaje:</b> %{x:.2f}%%<extra></extra>",
    customdata=df_percent[['Delito']]
)
fig.show()
```

Barra Comparativa

El gráfico de barras comparativa se utiliza para comparar multiples variables entre categorías, esto mediante barras de diferentes variables agrupadas lado a lado facilitando la compararación directa entre variables.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico y Código




```python
import numpy as np
import matplotlib.pyplot as plt

link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)
filtro_incluir = (datos["Provincia"] == "CARTAGO") | (datos["Provincia"] == "HEREDIA") #Filtro para solo inlcuir a Cartago y Heredia
filtro_final = filtro_incluir & (datos["Provincia"] != "DESCONOCIDO") #Filtro para excluir a DESCONOCIDO
datos_filtrados = datos[filtro_final] #los valores se convierten en un data frame
provincias = datos_filtrados['Provincia'].value_counts()

pivot_df = pd.crosstab(index=datos_filtrados['Provincia'],
                       columns=datos_filtrados['Delito'])
pivot_df = pivot_df.loc[pivot_df.sum(axis=1).sort_values(ascending=False).index]

fig, ax = plt.subplots(figsize=(10, 6)) #Decir el tamaño y ancho de las barras

bar_width = 0.12 #separación de las barras
x = np.arange(len(pivot_df.index)) #asigna un número para los delitos
num_sub_cats = len(pivot_df.columns) #por medio del número asignado se escoge una posición

offset_group_start = x - (bar_width * num_sub_cats / 2) + (bar_width / 2) #esto nos muestra el "ancho" que van a tener las barras

COLORES_DELITOS =["#e0f3db", "#a2d2a4", "#7bba91", "#559f80", "#2e846f", "#006a5e"][:num_sub_cats]

for i, sub_cat in enumerate(pivot_df.columns):
    ax.bar(
        offset_group_start + (i * bar_width),
        pivot_df[sub_cat],
        width=bar_width,
        label=sub_cat,
        color=COLORES_DELITOS[i]
    )
ax.set_xlabel("Provincias") #Leyenda del eje x
ax.set_ylabel("Cantidad de Delitos") #Leyenda del eje y
ax.set_title("Comparativa de Delitos entre Cartago y Heredia", loc="left") #Título del gráfico
ax.set_xticks(x) #establecer donde marcar las líneas
ax.set_xticklabels(pivot_df.index)
ax.set_axisbelow(True) #lineas del fondo
ax.grid(axis='y', zorder=0) #lineas por debajo de las barras
#donde se muestra los delitos y el color que los representa
ax.legend(title="Tipo de Delito",
          bbox_to_anchor=(1.01, 1),
            loc='upper left')

plt.show()
```

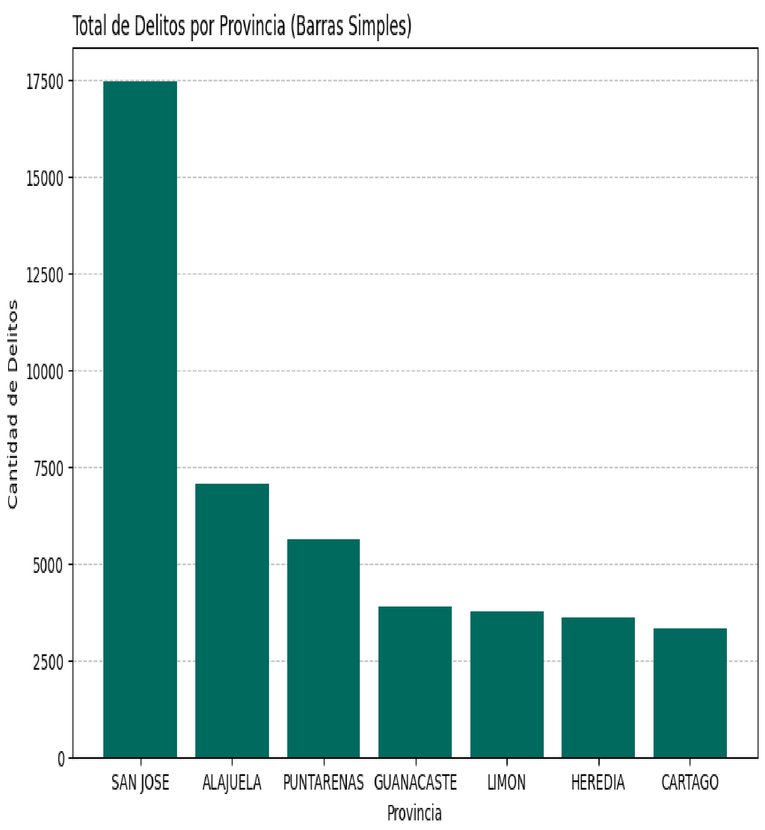
Gráficos Estadísticos: Barras Simples y Compuestas

Sebastián Calvo Solano, Isaac Rodríguez
Colegio Universitario de Cartago

Barra Simple

El gráfico de barras simples se utiliza para comparar una variable entre diferentes categorías, estas sirven para representar una sola variable y permiten una comparación directa y rápida, estas pueden ser horizontales o verticales.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico



Código

Código utilizado realizar el gráfico en Python:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

filtro = datos["Provincia"] != "DESCONOCIDO"  #los valores se transforman en booleanos y excluye DESCONOCIDO
datos_filtrados = datos[filtro] #los valores se convierten en un data frame
provincias_conteo = datos_filtrados['Provincia'].value_counts()
provincias_ordenadas = provincias_conteo.index.tolist()

fig, ax = plt.subplots(figsize=(10, 6)) #Figura de las barras

ax.bar(provincias_ordenadas, provincias_conteo.values, color='#006a5e', zorder=3)
#Colores de la barra

ax.set_title("Total de Delitos por Provincia (Barras Simples)", loc='left') #Título del gráfico
ax.set_xlabel("Provincia") #Leyenda para el eje X
ax.set_ylabel("Cantidad de Delitos") #Leyenda para el eje Y

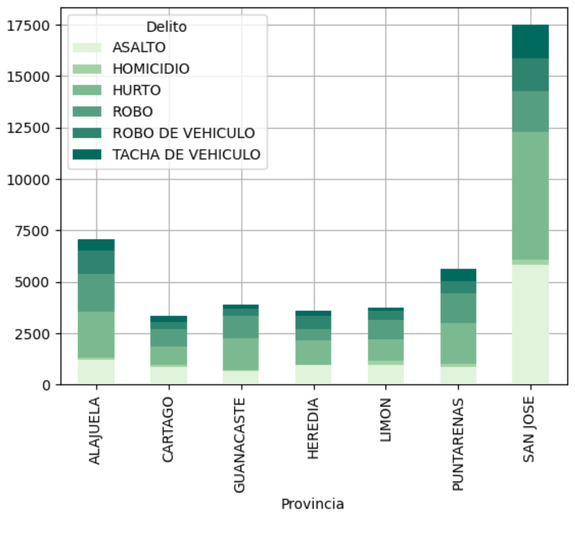
ax.set_axisbelow(True)
ax.grid(axis='y', linestyle='--', alpha=0.7, zorder=0) #Líneas del fondo

plt.show()
```

Barra Compuesta

El gráfico de barras compuestas permite dividir la información y mostrar los segmentos que la componen, cada elemento es una subcategoría, esto puede servir para mostrar el rango de edad de las personas que viven por provincias.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico y Código




```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

filtro = datos["Provincia"] != "DESCONOCIDO" #los valores se transforman en booleanos y excluye DESCONOCIDO
datos_filtrados = datos[filtro] #los valores se convierten en un data frame
provincias = datos_filtrados['Provincia'].value_counts()

pivot_df = pd.crosstab(index=datos_filtrados['Provincia'], #Las barras se van a mostrar por provincias (eje x)
                       columns=datos_filtrados['Delito']) #Las barras se se alzan por medio de los delitos (eje y)

num_delitos = len(pivot_df.columns) #Muestra la cantidad de tipos de delitos que hay (en este caso 6)

COLORES_DELITOS =["#e0f3db", "#a2d2a4", "#7bba91", "#559f80", "#2e846f", "#006a5e"][:num_delitos]
#Se le asigna un color a cada delito (gracias a num_delitos)

pivot_df.plot.bar(stacked=True, #Genera los bloques de división
                  grid=True, #Genera los cuadros que hay en el fondo
                  color=COLORES_DELITOS, #Los colores asignados anteriormente
                  zorder=3) #Las barras están por encima de los cuadros del fondo

plt.show()
```

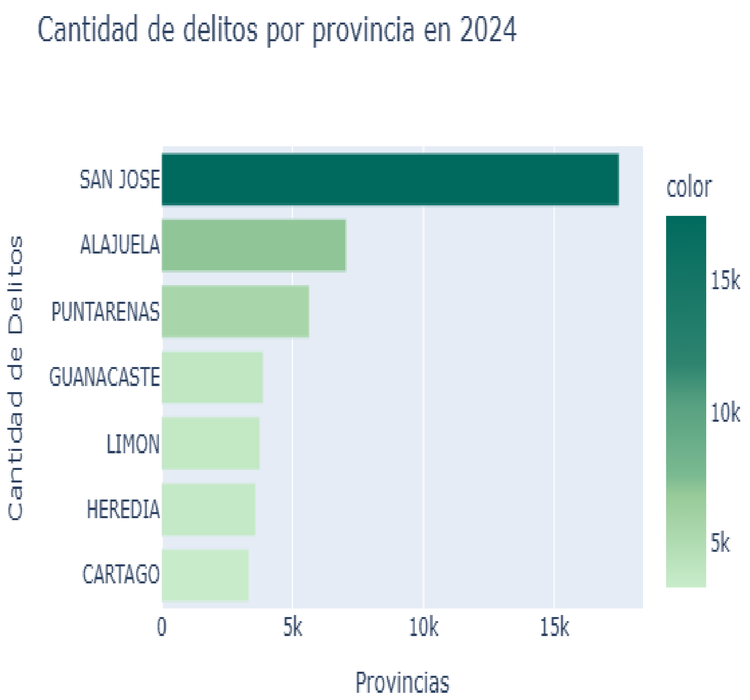
Gráficos Estadísticos: Barras Horizontales y Verticales

Sebastián Calvo Solano, Isaac Rodríguez
Colegio Universitario de Cartago

Barra Horizontal

El gráfico de barras horizontales se utiliza para mostrar datos cualitativos o geográficos, se coloca de menor a mayor donde la barra mas larga se posiciona arriba y en caso de haber una barra "otros" se coloca en la parte inferior.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico



Código

Código utilizado realizar el gráfico en Python:

```python
link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

filtro = datos["Provincia"] != "DESCONOCIDO" #los valores se transforman en booleanos y excluye DESCONOCIDO
datos_filtrados = datos[filtro] #los valores se convierten en un data frame
provincias = datos_filtrados['Provincia'].value_counts()
orden_provincias = provincias.index.tolist() #Ordenar las provincias en orden de delitos para mayor comodidad

#Gráfico de Barras Horizontal
fig = px.bar(
        x=provincias.values, #Valores del eje x (en este caso las provincias)
        y=provincias.index,  #Valores del eje y (en este caso los delitos)
        orientation= 'h', #Para poder poner las barras en modo horizonal hay que poner la H
        height=400, #Grosor de las barras
        title="Cantidad de delitos por provincia en 2024", #Título del gráfico
        color=provincias.values, #Colores utilizados que se ajustan a la cantidad de delitos de la provincia
        color_continuous_scale=[(0.00, "#c8ecca"),   (0.25, "#97ca99"),
                                (0.30, "#7bba91"), (0.50, "#559f80"),
                                (0.60, "#2e846f"),  (1.00, "#006a5e")],
        category_orders={"y": orden_provincias} # Ordena el eje y por cantidad de delitos
)

fig.update_traces( #Al ser hecho en "plotly" es un gráfico dinámico lo cual esto es para configurar el hover
    hovertemplate="<b>Provincia:</b> %{y}<br>" + #Nos muestra el nombre de la provincia
                  "<b>Delitos:</b> %{x:,}<br>" + #Nos muestra la cantidad exacta de delitos
                  "<extra></extra>"
)

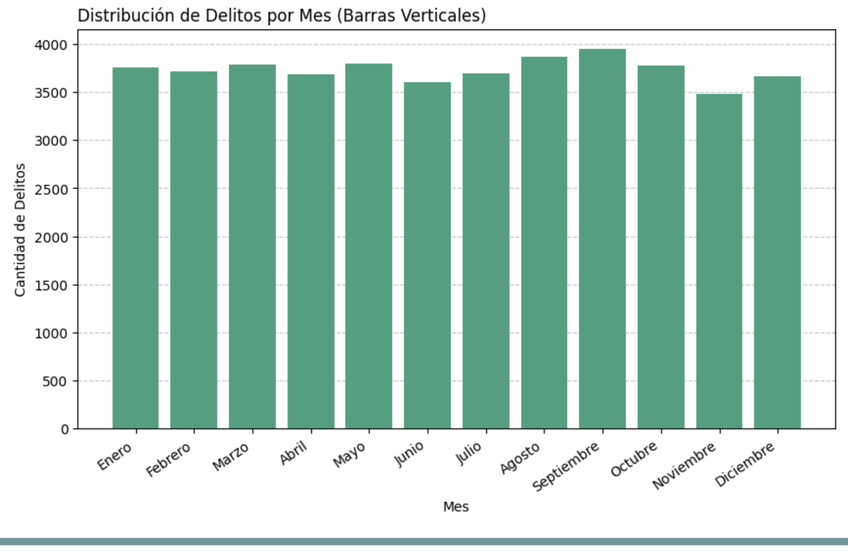
fig.update_xaxes(title_text='Provincias') #Leyenda del eje x
fig.update_yaxes(title_text='Cantidad de Delitos') #Leyenda del eje y

fig.show()
```

Barra Vertical

El gráfico de barras verticales se utiliza para mostrar datos cuantitativos discretos o cronológicos, estas mantienen un orden temporal o numérico y deja el mismo espacio entre barras para una mejor consistencia visual.
Los datos utilizados para realizar los gráficos fueron obtenidos del dataset del oij:
https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024

Gráfico y Código




```python
import pandas as pd #Librería que permite lectura de datos en línea
import numpy as np #Librería que permite una mejor lectura de datos y con menos consumo
import matplotlib.pyplot as plt #Librería para realizar el gráfico

#Código de optención de datos
link = 'https://sitiooij.poder-judicial.go.cr/phocadownload/Estadisticas/estadsticaspoliciales2024.xls'
datos = pd.read_excel(link)

datos["Fecha"] = pd.to_datetime(datos["Fecha"], errors='coerce') #Se obtiene los datos por fecha
datos['Nombre_Mes'] = datos["Fecha"].dt.month_name(locale='es_ES') #Se crea una categoría para agregar el nombre del mes
datos['Numero_Mes'] = datos["Fecha"].dt.month #Se crea una categoría para vincular el número del mes
datos_limpios = datos.dropna(subset=['Nombre_Mes'])
#Esto se hace ya que en el datset solo está la fecha del delito

#Se combinan los nombres de los meses con el de los números
conteo_mensual = datos_limpios.groupby(['Numero_Mes', 'Nombre_Mes']).size().reset_index(name='Total_Delitos')
conteo_mensual = conteo_mensual.sort_values(by='Numero_Mes')
meses = conteo_mensual['Nombre_Mes']
delitos_mes = conteo_mensual['Total_Delitos']

fig, ax = plt.subplots(figsize=(10, 6)) #Decir el tamaño y ancho de las barras

ax.bar(meses, delitos_mes, color='#559f80', zorder=3)
#Se especifica de donde se obtienen los datos, cual es el eje x, el eje y el color de las barras

#Se agrega el título y las leyendas
ax.set_title("Distribución de Delitos por Mes (Barras Verticales)", loc='left')
ax.set_xlabel("Mes")
ax.set_ylabel("Cantidad de Delitos")

#Esto es para una mejor lectura del nombre de los meses
ax.set_axisbelow(True)
ax.grid(axis='y', linestyle='--', alpha=0.7, zorder=0)
fig.autofmt_xdate(rotation=35)

plt.show()
```